

Лабораторна робота 4. Моніторинг в Kubernetes

Мета: Розгорнути стек моніторингу (Prometheus, Loki, Grafana, Alloy) у Kubernetes-кластері та налаштувати збір метрик і логів з todo-app.

Передумови

Перед початком роботи переконайтесь, що на вашій машині встановлено:

- Виконана лабораторна робота 2 (готові Docker-образи для фронтенду та бекенду)
- Встановлений Minikube (<https://minikube.sigs.k8s.io/docs/start/>)
- Встановлений kubectl (<https://kubernetes.io/docs/tasks/tools/>)
- Встановлений Helm - пакетний менеджер Kubernetes (<https://helm.sh/docs/intro/install/>).
- Minikube запущений з достатніми ресурсами:
`minikube start --cpus=4 --memory=4096`

Система

Ви працюватимете з готовим 3-рівневим веб-застосунком (todo-app), який складається з трьох компонентів:

- Frontend: React-застосунок, інтерфейс користувача.
- Backend: Python-сервер (FastAPI), який обробляє запити та працює з даними.
- База даних: PostgreSQL.

Репозиторій: https://github.com/igoris/cloud_computing_lab1

Система моніторингу буде складатись з наступних компонентів:

- Prometheus – збирає метрики (числові показники: CPU, RAM, кількість Pods).
- Loki – зберігає та індексує логи контейнерів.
- Grafana Alloy – агент, який читає логи з вузлів і надсилає їх у Loki. Замінює Promtail, який deprecated з 2026 року.
- Grafana – веб-інтерфейс для дашбордів, графіків та перегляду логів.

Разом із Темпро (трейси) та Mimir (масштабоване сховище метрик) ця комбінація відома як LGTM-стек від Grafana Labs. Mimir тут заміщений Prometheus (він легший для Minikube, а знання PromQL повністю переноситься на Mimir).

Завдання 1. Розгортання стеку моніторингу

Зверніть увагу, якщо в команді зустрічаються назви, написані курсивом, потрібно їх змінити на власні.

Створіть окремий namespace для моніторингу, щоб не змішувати ресурси з застосунком:

```
kubectl create namespace monitoring
```

Prometheus

Додайте репозиторій та встановіть Prometheus через Helm. Використаємо чарт kube-prometheus-stack — він одразу розгортає Prometheus Operator, Prometheus, Alertmanager, node-exporter та kube-state-metrics:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

```
helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --set grafana.enabled=false
```

Grafana буде встановлена окремо.

Перевірте статус Pods:

```
kubectl get pods -n monitoring
```

Дочекайтесь, поки всі Pods будуть у статусі Running (кілька хвилин).

Для Loki, Alloy та Grafana будемо використовувати values-файли замість --set-параметрів. Values-файл – це YAML з конфігурацією Helm. Values-файл можна закомітити в Git, переглянути в code review, повторити на іншому кластері.

Loki

Додайте репозиторій Grafana:

```
helm repo add grafana https://grafana.github.io/helm-charts
```

```
helm repo update
```

Завантажте value-файл для Loki:

<https://gist.github.com/igoris/652915a919ef9b46b4f3a906ca3d3019>

Встановіть Loki:

```
helm install loki grafana/loki --namespace monitoring -f loki-values.yaml
```

Дочекайтесь, поки pod loki-0 перейде в статус Running. Ініціалізація триває близько 30 секунд. Протягом цього часу liveness/readiness-проби можуть показувати Warning, це нормально.

Alloy

Alloy запускається як DaemonSet на кожному вузлі, читає логи контейнерів і надсилає їх у Loki. Конфігурація описується мовою River.

Завантажте файл конфігурації:

<https://gist.github.com/igoris/192a0e551d37f591e0c429846fcd28f8>

В цьому файлі pipeline складається з чотирьох компонентів: discovery.kubernetes шукає поди → discovery.relabel додає мітки (namespace, app) → loki.source.kubernetes читає логи → loki.write надсилає в Loki.

Встановіть Alloy:

```
helm install alloy grafana/alloy --namespace monitoring -f alloy-values.yaml
```

Grafana

Завантажте файл

<https://gist.github.com/igoris/2714ce4c3e565fea885548c12adaf49c>

У секції persistence включено збереження даних, щоб дашборди переживали перезавантаження Pod або Minikube (minikube stop + minikube start). minikube delete знищує VM разом з даними.

У секції datasources описано два джерела даних; Grafana підхопить їх автоматично при старті, додатково в UI їх налаштовувати не потрібно.

Встановіть Grafana:

```
helm install grafana grafana/grafana --namespace monitoring -f grafana-values.yaml
```

Отримайте доступ до Grafana через port-forward:

```
kubectl port-forward service/grafana 3001:80 -n monitoring
```

Перейдіть у Connections → Data sources. Переконайтесь, що Prometheus та Loki показують статус Working (кнопка Test на сторінці datasource).

У loki-values.yaml зараз persistence вимкнений – логи пропадуть при перезапуску Minikube. Додайте конфігурацію persistence у loki-values.yaml. В Loki persistence налаштовується не на верхньому рівні values, як у Grafana, а всередині блоку singleBinary. Застосуйте зміни:

```
helm upgrade loki grafana/loki --namespace monitoring -f loki-values.yaml
```

Перевірте, що з'явився новий claim зі статусом Bound:

```
kubectl get pvc -n monitoring
```

Дайте відповіді на питання:

1. Чим відрізняються метрики від логів? Коли що використовувати?
2. Чому Loki індексує лише labels, а не повний текст логів? Які переваги і обмеження цього підходу?
3. Чому моніторинг розгорнуто в окремому namespace, а не разом із застосунком у default?

Додайте файли в репозиторій і додайте в звіт посилання на комміт, бранч чи пулл реквест з цими файлами.

Завдання 2. Дослідження метрик та логів

Метрики

У Grafana Explore з data source Prometheus виконайте наступні запити та опишіть у звіті, що показує кожен з них, додайте скріншоти:

```
count(kube_pod_info)
```

```
sum(rate(container_cpu_usage_seconds_total{namespace="default"}[5m])) by (pod) * 100
```

```
sum(container_memory_working_set_bytes{namespace="default"}) by (pod) / 1024 / 1024
```

Логи

У Grafana Explore оберіть data source Loki, виконайте наступні запити:

```
{namespace="default", app="backend"}
```

```
{namespace="default", app="backend"} |= "POST"
```

Опишіть, що вони повертають.

Створіть, відредагуйте та видаліть кілька записів у todo-app через браузер. Знайдіть відповідні HTTP-запити в логах бекенду. Додайте в звіт приклади логів для POST, GET, DELETE-запитів.

Дашборд

Створіть новий дашборд у Grafana з наступними панелями:

- CPU (%) – time series з PromQL-запитом для CPU по pods у namespace default. У налаштуваннях панелі в полі Unit оберіть Percent (0-100)
- Пам'ять (MiB) – time series з PromQL-запитом для пам'яті. У полі Unit оберіть Mebibytes
- Кількість Pods – stat-панель із запитом на кількість запущених подів
- Логи бекенду – logs-панель з LogQL-запитом для логів бекенду

Збережіть дашборд. Додайте скріншот у звіт.

Моніторинг

Створіть навантаження на бекенд у окремому терміналі:

```
kubectl run loadtest --rm -it --image=busybox -- sh -c "while true; do wget -q -O- http://service_name:port/api/todos; done"
```

Дочекайтесь, поки на дашборді з'явиться зростання CPU та пам'яті бекенду (1–2 хвилини). Додайте скріншот в звіт.

Поки навантаження йде, видаліть Pod бекенду:

```
kubectl delete pod pod_name
```

Спостерігайте за дашбордом і Loki. Додайте в звіт скріншоти графіків з моментом падіння та відновлення Pod.

Дайте відповіді на питання:

1. Чи видно момент падіння Pod на графіку CPU? Як саме – провал, стрибок, зникнення? Чи змінилось ім'я Pod?
2. Скільки часу минуло від видалення Pod до появи метрик з нового Pod?
3. Якби цього моніторингу не було (як у лабораторних 1–3), як би ви дізнались про падіння Pod?

Список літератури:

1. <https://prometheus.io/docs/prometheus/latest/querying/basics/>
2. <https://grafana.com/docs/loki/latest/query/>
3. <https://grafana.com/docs/alloy/latest/>
4. <https://grafana.com/docs/grafana/latest/panels-visualizations/>